WEB APPLICATION DEVELOPMENT
LABORATORY MANUAL

Course: Web Application Development

Department of Computer Engineering

Student Name: _____
Roll No: _____
Academic Year: _____

Index

1. HTML5 Web Page Design

2. HTML and CSS3 Portfolio Website

3. Figma Workspace

4. Reusable Components in Figma

5. Interactive Prototype

6. JavaScript Programming

7. Node.js HTTP Server

8. Node.js File System

9. Email Automation using Node.js

10. MongoDB Setup

11. MongoDB CRUD Operations

12. MongoDB Relationships

13. MongoDB Aggregation and Indexing

14. MongoDB Atlas

15. Git GitHub Jenkins and Tomcat

Experiment 1

Aim

To design a webpage using HTML5 elements such as headings, paragraphs, lists, images, tables, hyperlinks, iframe, audio, and basic formatting tags.

Learning Objectives

After completing this experiment, students will be able to:

- Understand the structure of an HTML document.
- Create webpages using HTML elements.
- Apply text formatting.
- Insert images, tables, hyperlinks, and multimedia content.
- Design a basic static webpage.

Tools Required

- Visual Studio Code
- Web Browser (Google Chrome)

Theory

HTML (HyperText Markup Language) is used to create the structure of webpages. HTML documents contain different elements represented using tags.

Basic structure:

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>

<body>

Webpage Content

</body>
</html>
```

Procedure

Step 1: Create Project Folder

1. Open **Visual Studio Code**.
2. Select:

```
File → Open Folder
```

3. Create a new folder:

```
HTML_Practical
```

4. Inside this folder create a file:

```
index.html
```

Step 2: Write HTML Code

Open **index.html** and add the following code:

```
<!DOCTYPE html>
<html>

<head>

<title>
HTML Webpage Design
</title>

</head>


<body>


<h1>
Welcome to Web Application Development
</h1>


<p>
HTML is used to create the structure of websites.
</p>



<h2>
```

```
Programming Languages
</h2>


<ul>

<li>HTML</li>
<li>CSS</li>
<li>JavaScript</li>

</ul>



<h2>
Student Details
</h2>


<table border="1">


<tr>

<th>Name</th>

<th>Department</th>

<th>Year</th>

</tr>


<tr>

<td>
Rahul
</td>

<td>
Computer Engineering
</td>

<td>
Second Year
</td>

</tr>


</table>



<h2>
Image Example
</h2>
```

```
<img src="image.jpg"
width="300"
height="200">



<h2>
Useful Website</h2>


<a href="https://www.google.com">

Click Here to Visit Google

</a>



<h2>
Audio Example
</h2>


<audio controls>

<source src="audio.mp3">

</audio>



<h2>
Iframe Example
</h2>


<iframe
src="https://www.wikipedia.org"
width="500"
height="300">

</iframe>


</body>

</html>
```

Step 3: Add Resources

Inside the project folder add:

```
HTML_Practical

├── index.html
├── image.jpg
└── audio.mp3
```

Step 4: Execute Program

1. Open `index.html` in VS Code.
2. Install **Live Server Extension** in VS Code.
3. Right-click on `index.html`.
4. Select:

```
Open with Live Server
```

5. The webpage opens in the browser.

Output

The webpage displays:

- Heading and paragraph
- List of programming languages
- Student information table
- Image
- Hyperlink
- Audio player
- Embedded webpage using iframe

Output Screenshot

[Paste experiment output screenshot here]

Result

The experiment was successfully completed.

Experiment 2

Aim

To design a personal portfolio webpage using HTML5 and CSS3 properties such as colors, borders, padding, gradients, box model, positioning, and external CSS.

Tools Required

- Visual Studio Code
- Google Chrome Browser

Project Structure

Create the following folder:

```
Portfolio_Website
│
├── index.html
│
├── style.css
│
└── images
```

Procedure

Step 1: Create HTML File

Create:

`index.html`

Add the following code:

```
<!DOCTYPE html>

<html>

<head>
```

```
<title>
My Portfolio
</title>


<link rel="stylesheet"
href="style.css">


</head>


<body>


<header>

<h1>
Akshay Patil
</h1>


<p>
Computer Engineering Student
</p>


</header>



<section>

<h2>
About Me
</h2>


<p>

I am learning Web Application Development
using HTML, CSS and JavaScript.

</p>


</section>




<section>


<h2>
My Skills
</h2>
```

```
<ul>

<li>HTML5</li>

<li>CSS3</li>

<li>JavaScript</li>

<li>Node.js</li>

</ul>


</section>




<section>


<h2>
Projects
</h2>



<table>


<tr>

<th>
Project Name
</th>


<th>
Description
</th>


</tr>


<tr>

<td>
Online Shopping Website
</td>


<td>
E-commerce Application
</td>
```

```
</tr>


</table>


</section>



<section>


<h2>
Contact Form
</h2>



<form>


<label>
Name
</label>

<input type="text">


<br><br>


<label>
Email
</label>


<input type="email">


<br><br>


<button>
Submit
</button>



</form>


</section>
```

```
</body>

</html>
```

Step 2: Create CSS File

Create:

```
style.css
```

Add:

```
body
{

font-family: Arial;

background-color:#f2f2f2;

}



header
{

background:linear-gradient(
to right,
blue,
skyblue);

color:white;

text-align:center;

padding:30px;

}



section
{

background:white;

margin:20px;

padding:20px;

border-radius:10px;
```

```
}


table
{

width:100%;

border-collapse:collapse;

}


th,td
{

border:1px solid black;

padding:10px;

}


button
{

background:blue;

color:white;

padding:10px;

border:none;

}
```

Step 3: Run Application

1. Open `index.html`.
2. Right-click.
3. Select:

```
Open with Live Server
```

Output

A personal portfolio webpage containing:

- Profile section
- Skills section
- Project details
- Contact form
- CSS styling effects

(Attach screenshot)

Result

The experiment was successfully completed.

Experiment 3

Aim

To understand the Figma workspace, menus, toolbars, layers, frames, and basic design tools used for creating user interface designs.

Learning Objectives

After completing this experiment, students will be able to:

- Understand the Figma interface.
- Create frames for UI design.
- Use basic design tools.
- Manage layers and objects.
- Create simple UI layouts.

Software Requirements

- Web Browser (Google Chrome recommended)
- Figma Account
- Internet Connection

Theory

What is Figma?

Figma is a cloud-based UI/UX design tool used for creating wireframes, prototypes, and user interface designs.

It provides:

- Real-time collaboration
- Vector design tools
- Components and reusable assets
- Prototyping features
- Design sharing

Figma Workspace Components

Component	Description
Toolbar	Contains design tools
Canvas	Area where designs are created
Layers Panel	Displays objects and hierarchy
Properties Panel	Shows object properties
Assets Panel	Contains reusable components

Procedure

Step 1: Open Figma

1. Open browser.
2. Visit Figma website.
3. Login using your account.
4. Click:

```
New Design File
```

Step 2: Create a Frame

1. Select **Frame Tool** from toolbar.
2. Choose:

```
Mobile → iPhone 14
```

3. A mobile screen frame is created.

Step 3: Add Shapes

1. Select Rectangle Tool.

Shortcut:

```
R
```

2. Draw a rectangle.
3. Modify:

- Width
- Height
- Fill color
- Border radius

Step 4: Add Text

1. Select Text Tool.

Shortcut:

```
T
```

2. Click on canvas.
3. Enter:

```
Welcome to My App
```

4. Change:

- Font size
- Font style
- Alignment

Step 5: Manage Layers

1. Open Layers Panel.
2. Rename objects:

Example:

```
Rectangle → Header Background

Text → Heading
```

3. Arrange objects using drag and drop.

Step 6: Save Design

Figma automatically saves the design.

Click:

```
Share → Copy Link
```

to share the design.

Output

A basic mobile UI layout containing:

- Header section
- Text elements
- Shapes
- Organized layers

(Attach screenshot of Figma design)

Result

The experiment was successfully completed.

Experiment 4

Aim

To apply UI design principles and create reusable components using Figma components and libraries.

Learning Objectives

Students will learn:

- Basic UI design principles.
- Component creation.
- Asset management.
- Reusing design elements.

Software Requirements

- Figma

Theory

UI Design Principles

1. Consistency

Maintaining similar colors, fonts, and layouts throughout an application.

2. Alignment

Arranging elements properly to improve readability.

3. Hierarchy

Highlighting important information using size and placement.

4. Simplicity

Creating clean and easy-to-use interfaces.

Procedure

Step 1: Create New Frame

1. Open Figma.
2. Create mobile frame.

Example:

```
Mobile → Android
```

Step 2: Create Button Component

1. Select Rectangle Tool.

Create:

```
Width: 200 px
Height: 50 px
```

2. Add text:

```
LOGIN
```

3. Select both objects.

Right-click:

```
Create Component
```

Shortcut:

```
Ctrl + Alt + K
```

Step 3: Create Multiple Instances

1. Duplicate component.
2. Change:

- Text
- Color
- Size

without modifying original component.

Step 4: Create Card Component

Create a card containing:

- Image placeholder
- Title
- Description
- Button

Select all elements.

Choose:

```
Create Component
```

Step 5: Create Design Library

1. Open Assets panel.
2. Publish components.
3. Reuse components in different screens.

Output

A UI design containing:

- Login button component
- Card component
- Reusable UI elements

(Attach screenshot)

Result

The experiment was successfully completed.

Experiment 5

Aim

To create an interactive prototype using Figma by adding transitions, animations, and interactive states.

Learning Objectives

Students will be able to:

- Create interactive UI screens.
- Add prototype connections.
- Apply animations.
- Share designs for feedback.

Software Requirements

- Figma

Theory

Figma Prototype mode allows designers to connect multiple screens and simulate application behavior.

Prototype features:

- Screen navigation
- Button interactions
- Animations
- Transitions
- Overlay screens

Procedure

Step 1: Create Two Screens

Create:

Screen 1

```
Login Page
```

Contains:

- Username field
- Password field
- Login button

Screen 2

```
Home Page
```

Contains:

- Welcome message
- Dashboard cards

Step 2: Enable Prototype Mode

1. Select first screen.
2. Click:

```
Prototype
```

from right panel.

Step 3: Create Interaction

1. Select Login Button.
2. Drag connection arrow to Home Page.

Set:

```
Trigger:
On Click

Action:
Navigate To

Animation:
Smart Animate
```

Step 4: Add Animation

Choose:

```
Smart Animate
```

Options:

- Ease In
- Ease Out
- Slide In

Step 5: Test Prototype

Click:

```
Present ▶
```

Run the prototype.

Step 6: Share Prototype

Click:

```
Share → Copy Link
```

Output

Interactive prototype containing:

- Multiple screens
- Button interaction
- Screen transition
- Animation effects

(Attach prototype screenshot)

Result

The experiment was successfully completed.

Experiment 6

Aim

To develop JavaScript programs using variables, arrays, loops, functions, events, DOM manipulation, regular expressions, and form validation techniques.

Learning Objectives

After completing this experiment, students will be able to:

- Understand JavaScript syntax and programming concepts.
- Handle browser events.
- Manipulate HTML elements using DOM.
- Validate user input.
- Implement secure form handling.

Software Requirements

- Visual Studio Code
- Google Chrome Browser
- Live Server Extension

Theory

JavaScript

JavaScript is a client-side scripting language used to create dynamic and interactive webpages.

JavaScript can perform:

- Form validation
- Event handling
- Dynamic content updates
- Calculations
- DOM manipulation

Project Structure

Create a folder:

```
JavaScript_Practical
|
```

```
├── index.html
│
└── script.js
```

Procedure

Step 1: Create HTML File

Create:

`index.html`

Add the following code:

```
<!DOCTYPE html>
<html>

<head>

<title>
JavaScript Form Validation
</title>

</head>


<body>


<h1>
Student Registration Form
</h1>


<form onsubmit="return validateForm()">


<label>
Name:
</label>

<input type="text" id="name">


<br><br>


<label>
Email:
</label>

<input type="text" id="email">
```

```
<br><br>


<label>
Mobile Number:
</label>

<input type="text" id="mobile">


<br><br>


<button type="submit">
Submit
</button>


</form>


<p id="message"></p>


<script src="script.js"></script>


</body>

</html>
```

Step 2: Create JavaScript File

Create:

```
script.js
```

Add:

```
function validateForm()
{

let name =
document.getElementById("name").value;


let email =
document.getElementById("email").value;


let mobile =
```

```
document.getElementById("mobile").value;



let emailPattern =
/^[^\s@]+@[^\s@]+\.[^\s@]+$/;


let mobilePattern =
/^[0-9]{10}$/;



if(name=="")
{

alert("Enter your name");

return false;

}



if(!emailPattern.test(email))
{

alert("Enter valid email");

return false;

}



if(!mobilePattern.test(mobile))
{

alert("Enter valid mobile number");

return false;

}


document.getElementById("message").innerHTML =
"Form Submitted Successfully";


return false;

}
```

Step 3: Execute Program

1. Open `index.html`.
2. Right-click.
3. Select:

```
Open with Live Server
```

4. Enter student details.
5. Click Submit.

Output

The webpage displays:

- Registration form
- Validation messages
- Successful submission message

(Attach screenshot)

Result

The experiment was successfully completed.

Experiment 7

Aim

To create a simple web application using Node.js and demonstrate the use of Node.js core modules and HTTP module.

Learning Objectives

Students will learn:

- Installation and setup of Node.js.
- Creating a server using HTTP module.
- Handling client requests and responses.

Software Requirements

- Visual Studio Code
- Node.js

Theory

Node.js

Node.js is a JavaScript runtime environment that allows JavaScript execution outside the browser.

Features:

- Event-driven architecture
- Non-blocking I/O
- Fast server-side execution
- Built-in modules

Important Node.js Modules

Module	Purpose
HTTP	Create web servers
File System	File handling

Module	Purpose
URL	URL processing
Events	Event handling

Project Structure

Create:

```
Node_HTTP_Server
└── server.js
```

Procedure

Step 1: Open Terminal in VS Code

Go to:

```
Terminal → New Terminal
```

Check Node.js installation:

```
node -v
```

Step 2: Create JavaScript File

Create:

```
server.js
```

Step 3: Write Node.js Program

```
const http = require('http');


const server =
http.createServer((request,response)=>{


response.writeHead(200,
{
'Content-Type':'text/html'
```

```
});


response.write(
"<h1>Welcome to Node.js Server</h1>"
);


response.end();


});


server.listen(3000,
()=>{

console.log(
"Server running on port 3000"
);

});
```

Step 4: Run Application

Open terminal:

```
node server.js
```

Output:

```
Server running on port 3000
```

Open browser:

```
http://localhost:3000
```

Output

Browser displays:

```
Welcome to Node.js Server
```

(Attach screenshot)

Result

The experiment was successfully completed.

Experiment 8

Aim

To implement file handling operations using Node.js File System module and demonstrate URL and event modules.

Learning Objectives

Students will learn:

- Creating and reading files using Node.js.
- Handling URLs.
- Using Node.js events.

Project Structure

```
Node_File_System

├── app.js
└── sample.txt
```

Procedure

Step 1: Create app.js

```
const fs = require('fs');


// Creating file

fs.writeFileSync(
"sample.txt",
"Welcome to Node.js File System Module"
);


// Reading file

fs.readFile(
"sample.txt",
"utf8",
(error,data)=>{


if(error)

console.log(error);
```

```
else

console.log(data);


});
```

Step 2: Execute Program

Run:

```
node app.js
```

Output

Terminal displays:

```
Welcome to Node.js File System Module
```

Result

The experiment was successfully completed.

Experiment 9

Aim

To create a Node.js application for automated email sending.

Software Requirements

- Visual Studio Code
- Node.js
- Nodemailer Package

Theory

Nodemailer is a Node.js module used for sending emails from applications.

Procedure

Step 1: Create Project

Open terminal:

```
mkdir Email_Project

cd Email_Project
```

Initialize:

```
npm init -y
```

Install package:

```
npm install nodemailer
```

Step 2: Create mail.js

```
const nodemailer =
require("nodemailer");


let transporter =
nodemailer.createTransport({

service:"gmail",

auth:{
```

```
user:"your_email@gmail.com",

pass:"your_password"

}

});


let mailOptions={

from:"your_email@gmail.com",

to:"receiver@gmail.com",

subject:"Node.js Email Test",

text:"Email sent using Node.js"

};


transporter.sendMail(
mailOptions,
function(error,info){


if(error)

console.log(error);


else

console.log(
"Email sent successfully"
);


});
```

Step 3: Run Program

```
node mail.js
```

Output

Email is sent successfully.

Result

The experiment was successfully completed.

Experiment 10

Aim

To install MongoDB environment and perform basic database creation and deletion operations.

Software Requirements

- Visual Studio Code
- MongoDB Community Server
- MongoDB Compass

Theory

MongoDB is a NoSQL database that stores data in document format using BSON.

Features:

- Flexible schema
- High scalability
- Document-based storage

Procedure

Step 1: Start MongoDB

Open MongoDB Compass.

Connect:

```
mongodb://localhost:27017
```

Step 2: Create Database

Database Name:

```
StudentDB
```

Collection:

```
Students
```

Step 3: Insert Document

Insert:

```
{
"name":"Rahul",
"department":"Computer Engineering",
"year":2
}
```

Step 4: View Data

Click:

```
Find
```

to display records.

Output

MongoDB database with student collection is created successfully.

(Attach MongoDB Compass screenshot)

Result

The experiment was successfully completed.

Experiment 11

Aim

To perform Create, Read, Update, and Delete (CRUD) operations on MongoDB collections using Node.js.

Learning Objectives

After completing this experiment, students will be able to:

- Connect Node.js applications with MongoDB.
- Create and manage collections.
- Insert, retrieve, update, and delete documents.
- Use MongoDB queries.

Software Requirements

- Visual Studio Code
- Node.js
- MongoDB Server
- MongoDB Compass

Theory

MongoDB stores data in the form of documents.

Example:

```
{
"name":"Amit",
"department":"CSE",
"year":3
}
```

CRUD operations:

Operation	Description
Create	Insert new documents
Read	Retrieve documents

Operation	Description
Update	Modify existing documents
Delete	Remove documents

Project Structure

Create folder:

```
MongoDB_CRUD
```

```
├── app.js
└── package.json
```

Step 1: Install MongoDB Package

Open VS Code terminal:

```
npm init -y
```

Install MongoDB driver:

```
npm install mongodb
```

Step 2: Create app.js

```
const {MongoClient}=require("mongodb");


const url="mongodb://127.0.0.1:27017";


const client=new MongoClient(url);


async function main()
{
await client.connect();


console.log("MongoDB Connected");
```

```
const database =
client.db("StudentDB");


const collection =
database.collection("Students");



// CREATE

await collection.insertOne({

name:"Rahul",

department:"Computer Engineering",

year:2

});


console.log("Record Inserted");



// READ

const data =
await collection.find().toArray();


console.log(data);



// UPDATE

await collection.updateOne(

{name:"Rahul"},

{$set:{year:3}}

);


console.log("Record Updated");



// DELETE

await collection.deleteOne(

{name:"Rahul"}
```

```
);

console.log("Record Deleted");

}

main();
```

Step 3: Execute Program

Run:

```
node app.js
```

Output

Terminal displays:

```
MongoDB Connected

Record Inserted

Record Updated

Record Deleted
```

Result

CRUD operations were successfully performed on MongoDB using Node.js.

Experiment 12

Aim

To understand MongoDB data modelling and implement relationships between collections.

Learning Objectives

Students will learn:

- MongoDB document structure.
- Embedded documents.
- Referenced documents.
- One-to-one and one-to-many relationships.

Theory

MongoDB supports two methods for representing relationships:

1. Embedded Documents

Data is stored inside another document.

Example:

```
{
"name":"Student1",

"address":
{
"city":"Pune",
"state":"Maharashtra"
}

}
```

2. Referenced Documents

Documents are linked using Object IDs.

Example:

Student Collection:

```
{
"name":"Rahul",
"course_id":101
}
```

Course Collection:

```
{
"course_id":101,
"name":"Computer Engineering"
}
```

Procedure

Step 1: Create Database

Database:

```
CollegeDB
```

Collections:

```
Students

Courses
```

Step 2: Insert Course Data

```
{
"course_id":101,
"name":"Computer Engineering"
}
```

Step 3: Insert Student Data

```
{
"name":"Rahul",
"course_id":101
}
```

Step 4: Verify Relationship

Use MongoDB Compass:

1. Open database.
2. Select collection.
3. Click Find.

Output

Collections showing related student and course data.

(Attach MongoDB Compass screenshot)

Result

MongoDB relationships and data modelling techniques were successfully implemented.

Experiment 13

Aim

To perform advanced MongoDB operations using aggregation pipelines and indexes.

Learning Objectives

Students will learn:

- Data aggregation.
- Query optimization.
- Creating indexes.
- Managing large datasets.

Theory

Aggregation

Aggregation processes multiple documents and produces summarized results.

Example:

Calculate average marks:

```
db.students.aggregate([
{
$group:
{
_id:"$department",
averageMarks:
{
$avg:"$marks"
}
}
}
])
```

Index

Indexes improve query performance.

Example:

```
db.students.createIndex(
{
```

```
"name":1
}
)
```

Procedure

Step 1: Insert Sample Data

Collection:

```
Students
```

Documents:

```
{
"name":"Amit",
"department":"CSE",
"marks":85
}
```

Step 2: Perform Aggregation

Open MongoDB Shell:

```
use CollegeDB
```

Run:

```
db.Students.aggregate([
{
$group:
{
_id:"$department",

AverageMarks:
{
$avg:"$marks"
}

}
}
])
```

Step 3: Create Index

Run:

```
db.Students.createIndex(
{
name:1
}
)
```

Step 4: Check Index

```
db.Students.getIndexes()
```

Output

Aggregation result:

```
CSE : 85
```

Index created successfully.

Result

Advanced MongoDB operations using aggregation and indexing were performed successfully.

Experiment 14

Aim

To create a cloud-based MongoDB database using MongoDB Atlas and connect it with a Node.js application.

Learning Objectives

Students will learn:

- Creating MongoDB Atlas cluster.
- Connecting cloud database with applications.
- Performing database operations remotely.

Software Requirements

- Visual Studio Code
- Node.js
- MongoDB Atlas Account

Procedure

Step 1: Create MongoDB Atlas Account

1. Open MongoDB Atlas website.
2. Create account.
3. Create a new project.

Step 2: Create Cluster

Select:

```
Create → Free Cluster
```

Wait until deployment completes.

Step 3: Create Database User

Add:

```
Username
Password
```

Step 4: Obtain Connection String

Click:

```
Connect → Drivers
```

Copy connection string.

Example:

```
mongodb+srv://username:password@cluster.mongodb.net/
```

Step 5: Install MongoDB Package

In VS Code terminal:

```
npm install mongodb
```

Step 6: Create Connection Program

app.js

```
const {MongoClient}=require("mongodb");


const url=
"mongodb+srv://username:password@cluster.mongodb.net/";


const client=
new MongoClient(url);


async function connect()
{

await client.connect();
```

```
console.log(
"Connected to MongoDB Atlas"
);


}


connect();
```

Step 7: Run Program

```
node app.js
```

Output

```
Connected to MongoDB Atlas
```

Result

A Node.js application was successfully connected with MongoDB Atlas cloud database.

Experiment 15

Aim

To understand version control using Git, repository management using GitHub, and deployment using Apache Tomcat.

Learning Objectives

Students will learn:

- Git installation and commands.
- Creating repositories.
- Uploading projects to GitHub.
- Deploying web applications on Tomcat server.

Software Requirements

- Visual Studio Code
- Git
- GitHub Account
- Apache Tomcat Server

Part A: Git and GitHub

Step 1: Configure Git

Open VS Code Terminal:

```
git --version
```

Configure user:

```
git config --global user.name "Student Name"
git config --global user.email "student@email.com"
```

Step 2: Create Repository

Create project folder:

```
WebApplicationProject
```

Initialize Git:

```
git init
```

Step 3: Add Files

```
git add .
```

Step 4: Commit Changes

```
git commit -m "Initial project commit"
```

Step 5: Push to GitHub

Create repository on GitHub.

Connect repository:

```
git remote add origin repository_url
```

Push:

```
git push -u origin main
```

Part B: Apache Tomcat Deployment

Step 1: Install Tomcat

Download and install Apache Tomcat.

Start server.

Step 2: Deploy Application

Copy web application folder into:

```
Tomcat/webapps
```

Step 3: Run Application

Open browser:

```
http://localhost:8080
```

Part C: Jenkins Introduction

Jenkins Features

- Continuous Integration
- Automated testing
- Automated deployment

Basic workflow:

```
Code Commit
      |
      ↓
Jenkins Build
      |
      ↓
Testing
      |
      ↓
Deployment
```

Output

- Git repository created.
- Project uploaded to GitHub.
- Web application deployed on Tomcat.

(Attach screenshots)

Result

Version control using Git, repository management using GitHub, and basic deployment using Apache Tomcat were successfully performed.